

Efficient Real-Time Video Colorization on Low-End CPUs via Pruning and Quantization

Khondker Salman Sayeed

IQVIA

Dhaka, Dhaka, Bangladesh

Computer Science and Engineering
Bangladesh University of Engineering
and Technology

Dhaka, Dhaka, Bangladesh

salkhon050@gmail.com

Haz Sameen Shahgir

Computer Science and Engineering

University of California Riverside

Riverside, California, USA

sameen2080@gmail.com

Tamzeed Mahfuz

Computer Science and Engineering

Bangladesh University of Engineering
and Technology

Dhaka, Dhaka, Bangladesh

2105012@ugrad.cse.buet.ac.bd

Satak Kumar Dey

Computer Science and Engineering

Bangladesh University of Engineering
and Technology

Dhaka, Dhaka, Bangladesh

2105100@ugrad.cse.buet.ac.bd

M Saifur Rahman

Computer Science and Engineering

Bangladesh University of Engineering
and Technology

Dhaka, Dhaka, Bangladesh

mrahman@cse.buet.ac.bd



Figure 1: A scene from Charlie Chaplin's film "The Great Dictator" colored using our pruned and quantized U-Net.

Abstract

Grayscale video capture remains a popular, low-cost approach for security and surveillance-related tasks, especially on edge devices. We create a deep-learning solution to colorize grayscale videos in real-time without dedicated acceleration hardware such as GPUs.

We first trained EfficientNet-B7-based U-Net on a combination

of image and video datasets. We prune redundant parameters in the bottleneck layers of the trained neural network using weight-base pruning, followed by minimal training to recover performance. Finally, we quantize parts of the neural network which reduces model size and inference-time memory requirement. Our final optimized model achieves a 43.75% inference speed improvement and 30.6% model size reduction over the base model and can colorize videos at 6+ frames per second on low-end CPUs while maintaining a competitive CDC score of 0.0031 and PSNR of 19.¹



This work is licensed under a Creative Commons Attribution International 4.0 License.

NSysS '24, December 19–21, 2024, Khulna, Bangladesh

© 2024 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1158-9/24/12

<https://doi.org/10.1145/3704522.3704536>

CCS Concepts

• Computing methodologies → Reconstruction.

¹Source: <https://github.com/salkhon/video-colorization>

Keywords

Colorization, Pruning, CNN, Spatio-temporal Consistency

ACM Reference Format:

Khondker Salman Sayeed, Haz Sameen Shahgir, Tamzeed Mahfuz, Satak Kumar Dey, and M Saifur Rahman. 2024. Efficient Real-Time Video Colorization on Low-End CPUs via Pruning and Quantization. In *11th International Conference on Networking, Systems, and Security (NSysS '24)*, December 19–21, 2024, Khulna, Bangladesh. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/3704522.3704536>

1 Introduction

Black and white videos hold a unique historical and artistic value. However, the lack of color can limit their engagement and accessibility to a wider audience. Colorization breathes life into these videos, providing a more immersive and informative experience. Traditionally, video colorization has been a manual, time-consuming process requiring artistic expertise [9, 23, 39].

The recent explosion of deep learning techniques offers a compelling alternative for automating video colorization. Deep learning models, trained on vast datasets of colorized images and videos, can learn the complex relationships between luminance information and color. Videos can be colorized using image colorization models to color each frame [12, 20, 41]. However, video colorization presents unique challenges. The model must process and colorize each frame individually while maintaining temporal coherence across the video sequence. The model must understand the colorization task for a single frame and account for the flow of information and visual consistency throughout the video. The most recent methods take multiple reference images as input to produce a consistent coloring [15, 27, 40]. These approaches result in fairly complex model architectures and thus slower inference speed. There is a gap in the literature regarding improving the inference time of video colorization models to make them usable in real-time on consumer hardware. Our work navigates the trade-off between complex model architectures and inference speed.

We use the U-Net [35] architecture with EfficientNet B7 [37] as a feature extractor. After an initial round of training, we apply weight-based pruning to improve inference speed. We uncover a fundamental challenge in pruning convolution neural networks (CNN) with skip connection, namely that the channel dimensions of two consecutive CNN layers are interdependent, and pruning the earlier CNN layer necessitates pruning the later layer as well. This interdependency is more complex when multiple CNN layers are connected to the subsequent layer via residual skip connections [7]. Since EfficientNet and the U-Net architecture rely heavily on residual skip connections, we prune only the bottleneck layers. Afterward, we retrain the model to recover performance and quantize parts of the trained model to improve memory efficiency. Our optimized model delivers a 43.75% improvement in inference speed and a 30.6% reduction in model size compared to the base model. It can colorize videos at over 6 FPS on an Intel-i5-8350U, a low-end CPU while maintaining a competitive Color Distribution Consistency (CDC) score of 0.0031, which reflects how closely the color distribution matches the ground truth. Additionally, it achieves a Peak Signal-to-Noise Ratio (PSNR) of 19, indicating a strong fidelity in the quality of the colorized videos relative to the original.

2 Related Work

Image Colorization with Deep Learning: Deep learning methods for image colorization began with Zhang et al. [41], who used a convolutional neural network (CNN) to predict chrominance values in the LAB color space. This work was extended by Larsson et al. [20] and Iizuka et al. [12], incorporating deep features and semantic information to improve colorization quality. Zhang et al. [42] later introduced a semi-automatic method where user-provided color hints were propagated across images using learned priors. More recently, ChromaGAN [38] used a GAN-based approach to generate vibrant, natural colorization, while ColorFormer [16] introduced a hybrid-attention transformer model for colorization. DDcolor [18] further refined these techniques with dual decoders, but maintaining temporal consistency remains a challenge for video tasks.

Video Colorization: Video colorization adds the challenge of ensuring consistency across frames. Jampani et al. [15] and Liu et al. [27] worked on this issue, but Deep Video Color Propagation [31] was the first method to incorporate semantic information globally while propagating color locally across frames. Zhang et al. [40] introduced a more refined approach using reference images and previous frames to guide colorization. Other significant contributions include deep feature propagation [28], self-regularization [22], VCGAN [43], and DeepRemaster [11], which applies temporal source-reference attention networks.

Pruning and Quantization: Model pruning, first explored in Optimal Brain Damage [21] and Optimal Brain Surgeon [6], reduces model complexity by removing less important weights. Han et al. [5] introduced weight-based pruning, which was refined by Liu et al. [29] through network slimming. Structured pruning [8], dynamic pruning [4], and the lottery ticket hypothesis [2] further advanced the field. Quantization, used alongside pruning, reduces model size by lowering precision. Early attempts included binary [10] and ternary [24] quantization, while later approaches such as quantization-aware training [14] and post-training 4-bit quantization [1] significantly enhanced compression.

Our work applies these techniques to video colorization, balancing computational efficiency with color fidelity.

3 Background

3.1 Skip/Residual Connections

Skip connections, also known as residual connections, are an essential technique in deep learning that allows neural networks to bypass intermediate layers by adding the input of a layer directly to its output. Formally, for an input x , instead of only learning the transformation $f(x)$, a skip connection computes $y = f(x) + x$.

This helps preserve information from earlier layers and mitigates the *vanishing gradient problem* [13], where gradients become too small to update weights in deep networks effectively.

In tasks like video colorization, skip connections are particularly useful in architectures such as U-Net, where they bridge the encoder and decoder layers. The encoder compresses the input image, while the decoder reconstructs it. Skip connections help the decoder access high-resolution features from the encoder, allowing for better preservation of details like edges and textures.

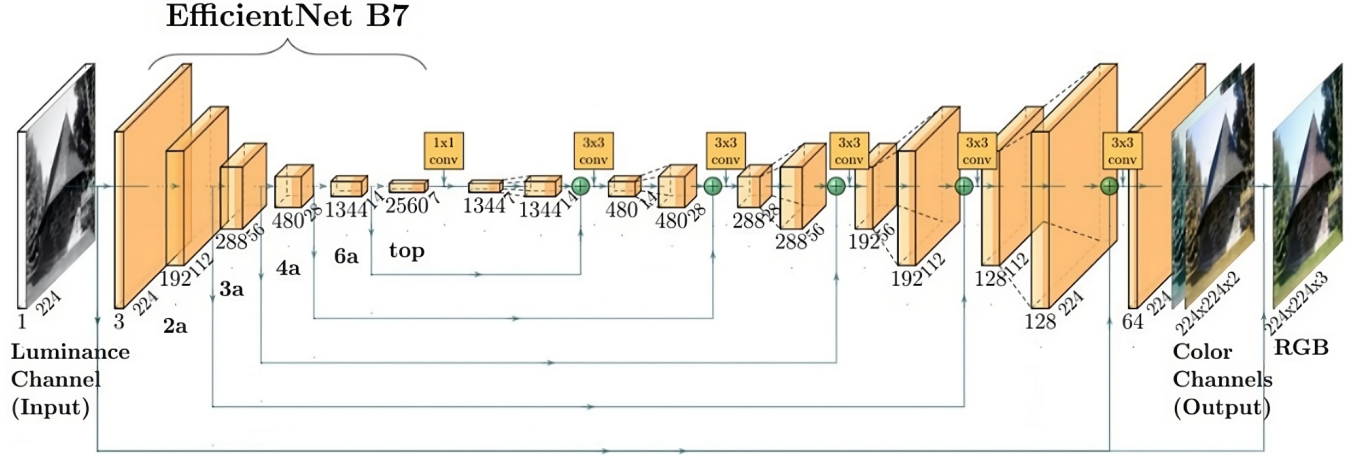


Figure 2: Proposed model architecture for video colorization with corresponding input and output. [25].

3.2 Pruning

Pruning is a widely used model compression technique that reduces the computational and memory costs of neural networks by removing unnecessary parameters. In convolutional neural networks (CNNs), this typically involves eliminating filters or channels from convolution layers, which reduces the number of operations and speeds up inference without a significant loss in accuracy [5]. Pruning can be either unstructured, where individual weights are removed, or structured, where entire filters or channels are pruned, the latter being more suitable for deployment on hardware due to its regularity.

Magnitude-based pruning is one of the most common techniques, where filters with the smallest weight magnitudes (L1 or L2 norms) are removed [29]. This method works effectively because many neural networks are over-parameterized, meaning that some filters learn redundant or irrelevant features. By pruning those with the least impact on performance, the model becomes smaller and faster while retaining most of its original accuracy.

3.3 Quantization

Quantization is a model compression technique that reduces the precision of a neural network's weights and activations, converting high-precision floating-point values (e.g., 32-bit) into lower precision representations (e.g., 8-bit integers). This significantly reduces the model's memory footprint and computational complexity, making it ideal for deployment in resource-constrained environments, such as edge devices or low-end CPUs.

During quantization, the network maps a range of floating-point values to a smaller set of fixed-point values. Using a representative input sample, the typical numerical range of a neuron is determined and the average of that range is used as an offset. For example, if a neuron is allowed to generate values from -16 to $+15$ using 4 bits but generates values ranging from 2 to 9, the quantization pipeline will take $\frac{2+9}{2} = 6$ as the constant offset and use a reduced range of -4 to $+3$ using only 2 bits. The original value can be represented as a sum of the offset and the 2 bit representation, allowing some loss in

precision. In practice, quantization reduced the model parameters from 32 bits to 8 bits with minimal impact on performance.

4 Methodology

4.1 Datasets

Selecting a training dataset is crucial in achieving robust performance for real-time video colorization. We initially considered the Places365 dataset [44] due to its extensive collection of images from diverse scenes. However, the sheer size of Places365 exceeded the computational resources available for our experiments. To address this challenge and maintain the desired dataset diversity, we opted for a combined approach utilizing two publicly available datasets: VQA-v2 [3] and DAVIS [34]. VQA-v2 offers a rich collection of images covering various real-world scenes, while DAVIS provides high-quality video sequences suitable for training in a video colorization context. Specifically, we employed the Train and Eval splits from the VQA-v2 dataset and the Train split from the DAVIS dataset for model training. We used the separate Eval split from the DAVIS dataset to evaluate the generalizability and performance of our colorization model. This ensures a robust evaluation process that utilizes unseen data during the final assessment.

Dataset	Size	Usage
Places [44]	2.5M Images	No
VQA-v2 [3]	80k Images	Train+Eval
DAVIS 2017 [34]	60 Videos	Train

Table 1: Datasets considered for training U-Net

4.2 Model Architecture

Following [25], we use a U-Net [35] architecture for image/video colorization. A typical U-Net architecture includes an encoder and a decoder, with multiple skip connections in intermediate layers. Figure 2 shows the overall architecture of the proposed model. The

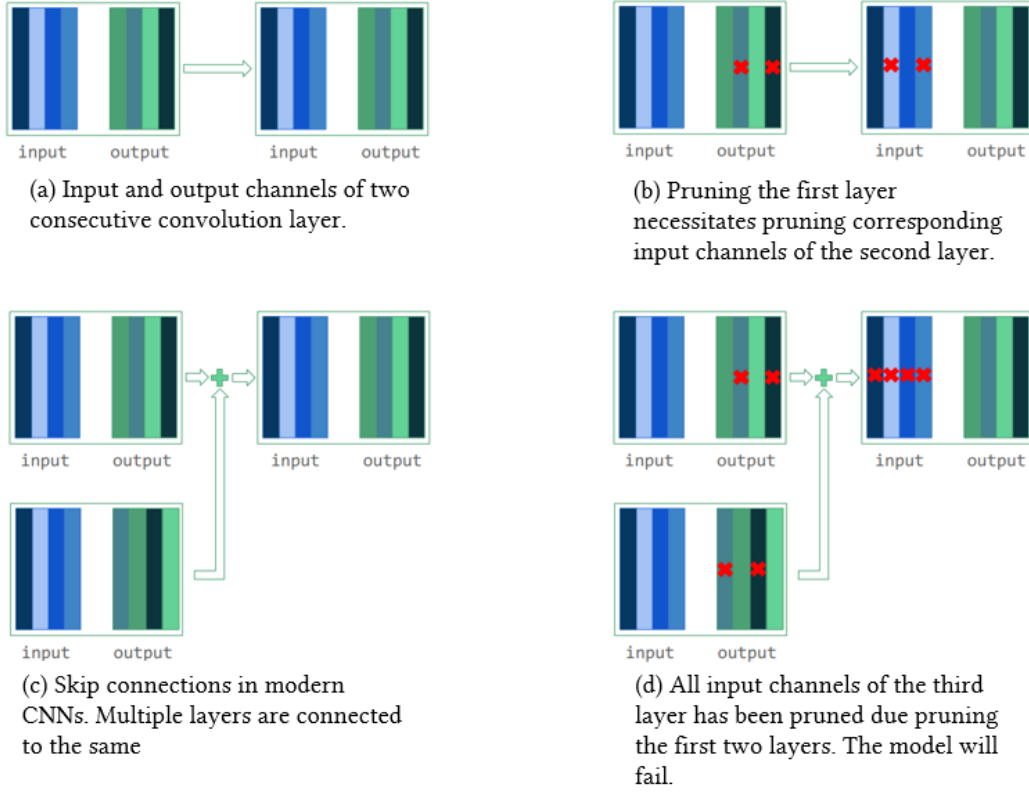


Figure 3: Skip connections limit pruning in Convolutional Neural Networks

Layer Location	Prune Ratio	Parameter Reduction
Pruning Squeeze and Excite blocks inside EfficientNet	50%	7.3%
Pruning Last Efficient-net and First Upsampling Block	100%	20%
Second Upsampling Block	30%	1.1%
Total		28.4%

Table 2: Pruning ratio for different convolution layers of the proposed U-Net architecture.

input to the model is the grayscale version of each image, and the ground truth is the RGB image. Intuitively, the model parameters must perform two implicit functions: object identification and color memorization/assignment. Firstly, the model must implicitly identify different objects in the grayscale image. Secondly, it must memorize the color of said objects during training and recall it during inference.

EfficientNet-B7 as U-Net Encoder: Previous research has demonstrated that utilizing a pretrained vision model as the encoder combined with a lightweight decoder achieves an optimal

balance between computational efficiency and colorization performance [25]. Due to our focus on real-time video colorization on low-end hardware, we pick EfficientNet-B7 [37] as the encoder layer. EfficientNet utilizes depthwise separable convolutions that reduce the computational complexity of convolution. EfficientNet also utilizes skip connections within itself which allows deeper CNN stacks by mitigating the vanishing gradient problem.

Lightweight U-Net Decoder: We use a 9-block decoder, where each block consists of a convolution layer, batch normalization, ReLU activation, and an upsampler. The upsampler uses the *nearest neighbor* upsampling algorithm and doesn't contain trainable parameters. We experiment with transposed convolution layers but notice no performance difference, corroborated in [25]. From the 82 million parameters in the complete U-Net, only 22% belong to the decoder while the rest belongs to the EfficientNet encoder.

Skip Connections: Skip connections between intermediate layers of the encoder and the decoder are integral architectural choices of the U-Net architecture [35]. Unlike ResNet [7], the purpose of U-Net skip connections is not to counter the vanishing gradient problem [13], rather it facilitates information transfer, especially structural information, from intermediate encoder layer representations to corresponding the decoder layers. Much of the structural information is collapsed when passing through the U-Net bottleneck layer and as such skip connections are vital to the effectiveness of U-Nets for image-to-image tasks such as segmentation and colorization.

4.3 Loss Functions

Inspired by [25], we used the pixel-wise Weighted Mean Squared Error loss function for our training. Losses incurred on pixels with rarer colors (as computed on the training dataset) are penalized more.

$$L_{pixel}(Y^{(i)}, \hat{Y}^{(i)}) = \frac{1}{2hw} \cdot \sum_{h,w} v(Y_{h,w}^{(i)}) \cdot \|Y_{h,w}^{(i)} - \hat{Y}_{h,w}^{(i)}\|_2^2 \quad (1)$$

In Equation 1, h, w are the dimensions of the image, $Y^{(i)}$ is the ground truth colored image tensor, $\hat{Y}_{h,w}^{(i)}$ is the model predicted tensor and v is the color weighting function. Since keeping track of all $256 \times 256 \times 256$ color combinations is computationally and memory intensive, we bin the colors into ~ 1 million bins after partitioning the color spectrum into 10 segments. We implement the lookup necessary for v using an efficient embedding layer² which is pre-computed and frozen during training.

To reduce color patches and color consistency across frames, we use perceptual loss [17] (See Equation 2). It measures the difference between the features extracted by a pre-trained CNN for the generated colorized frame and the ground truth color frame. Following [33], we use VGG11 [36] to compute perceptual loss. Perceptual loss and by extension, the VGG11 model is used only during training and the VGG11 model is used only for inference and never trained, keeping the pipeline efficient.

$$L_{VGG}(\phi^{(i)}, \hat{\phi}^{(i)}) = \frac{1}{2hw} \cdot \sum_{h,w} \|\phi_{h,w}^{(i)} - \hat{\phi}_{h,w}^{(i)}\|_2^2 \quad (2)$$

In Equation 2, ϕ is the feature tensor from VGG11 for our model's output, and $\hat{\phi}$ is the ground truth image's feature tensor from VGG11. After pilot experiments, we determined that unweighted addition yields the best colorization and hence, our combined loss is:

$$L(\phi^{(i)}, \hat{\phi}^{(i)}) = L_{pixel}(Y^{(i)}, \hat{Y}^{(i)}) + L_{VGG}(\phi^{(i)}, \hat{\phi}^{(i)}) \quad (3)$$

4.4 Pruning

Pruning has been widely studied in the context of convolutional neural networks (CNN). The authors of [5] establish a train-prune-retrain framework that we follow in our experiments. Each convolution layer of a CNN has multiple channels or filters that each learn different representations of the image. For example, in a JPEG format image, there are three channels Red, Green, and Blue. Each of the colors is a representation of the complete image and usually contains most of the salient information of the image. The per-channel representations learned by a convolution layer follow the same principle, although are less human-interpretable. Channels Convolution layers near the start of the model often learn to detect edges while upper layers detect entire objects [19].

convolution layers are often intentionally over-parameterized to improve training dynamics [2] and multiple channels converge during training and produce the same output. Since the output of the channels is averaged by the next convolution layer, they may also cancel out each other's contributions. Although redundant

parameters in the form of excess channels at each convolution layer make training easier, they do not contribute to downstream performance and rather slow down the inference since redundant computations are being performed.

As such, we choose to prune redundant channels at suitable convolution layers, which is a structured pruning strategy [5] and use magnitude-based pruning [21].

Algorithm 1: Pruning Algorithm for Deep Neural Networks

Input: $\mathcal{M}$: Model to prune, $\mathcal{L}$: List of layer indices to prune, $\mathcal{P}$: List of pruning ratios corresponding to each layer.

Output: Pruned model $\mathcal{M}$ with low-magnitude channels removed.

Function Prune

```

for  $(l, p)$  in  $(\mathcal{L}, \mathcal{P})$  do
     $\mathcal{L}_l \leftarrow \mathcal{M}.\text{layers}[l]$ ;
     $N_{\text{channels}} \leftarrow |\mathcal{L}_l.\text{channels}|$ ;
     $N_{\text{keep}} \leftarrow N_{\text{channels}} \times (1 - p)$ ;

     $\mathbf{m} \leftarrow [||\mathbf{w}_c||_1 \mid \mathbf{w}_c \in \mathcal{L}_l.\text{channels}]$ ;
     $\text{sorted\_indices} \leftarrow \text{sort}(\mathcal{L}_l.\text{channels}, \text{key} = \mathbf{m}, \text{reverse} = \text{True})$ ;

     $\text{pruned\_indices} \leftarrow \text{sorted\_indices}[: N_{\text{keep}}]$ ;

     $\mathcal{L}_l.\text{channels} \leftarrow \mathcal{L}_l.\text{channels}[\text{pruned\_indices}]$ ;
     $\mathcal{M}.\text{layers}[l] \leftarrow \mathcal{L}_l$ ;

     $\mathcal{L}_{l+1} \leftarrow \mathcal{M}.\text{layers}[l + 1]$ ;
     $\mathcal{L}_{l+1}.\text{channels} \leftarrow \mathcal{L}_{l+1}.\text{channels}[\text{pruned\_indices}]$ ;
     $\mathcal{M}.\text{layers}[l + 1] \leftarrow \mathcal{L}_{l+1}$ ;

return  $\mathcal{M}$ ;

```

How Skip Connections in CNNs limit Pruning: Algorithm 1 outlines the approach to pruning the trained U-Net. convolution layers are sequentially dependent since the channel dimension of the parameter weights of layer _{$i+1$} attend to the outputs of layer _{i} . In other words, if layer _{i} has n output channels, layer _{$i+1$} must have n input channels. Consequently, if any output channel of layer _{i} is pruned, the corresponding input channels of layer _{$i+1$} must be pruned as well. This is highly challenging in practice due to the skip connections present in modern CNNs, including EfficientNet and U-Net. If both layer _{a} and layer _{b} lead to layer _{c} , pruning the first two layers will mandate pruning twice as many input layers in layer _{c} . Since modern CNNs have multiple such skip connections attaching to any layer, almost all of the input channels of that layer will be pruned, resulting in a disconnected model. Figure 3 highlights this dilemma.

As such, we opt to prune only layers in our model where there are no skip connections, i.e. the bottleneck layers. Although a majority of the convolution layers cannot be pruned due to skip connections, pruning only the bottleneck layers provide significant performance improvements. As shown in Table 2, we determine target layers from pruning automatically and experimentally find optimal pruning ratios for each category of convolution layer. We

²<https://pytorch.org/docs/stable/generated/torch.nn.Embedding.html>



Figure 4: Pruning reduces image vibrancy which is recovered with retraining the pruned model.

did not experiment with tuning the pruning ratio of each individual layer but we expect it to yield further improvements. After pruning, we retrain the pruned model to recover performance.

4.5 Quantization

We use the PyTorch [32] native quantization pipeline for activation-aware quantization of the decoder layers of our trained U-Net. Using sample inputs, the pipeline determines the average range of values that a particular neuron generates and uses the mean of that range as the offset as explained in Section 3.3.

Pruning layers result in the model producing discolored images. This is fixed by fine-tuning the pruned model to adapt to its lost weight, as demonstrated in 4.

5 Experimental Setup

We trained our model on the images and frames from VQA-v2 [3] and DAVIS [3] for 10 epochs with a batch-size of 64, learning rate 10^{-3} using the AdamW [30] optimizer. After pruning, we train the model for a further 3 epochs and finally quantize the model. We use 256 random samples from the train set for activate-aware quantization of the U-Net decoder. We use the low-end Intel Intel-i5 8350U CPU for real-time inference.

6 Results and Discussion

We evaluated the performance of our proposed real-time video colorization model using the DAVIS [34] video dataset’s evaluation split.

6.1 Inference Speed

Table ?? compares the performance of the baseline model, pruned model, and pruned+quantized model in terms of FPS (frames per second), FPS gain, and model size. The baseline model achieves 4.54 FPS with a size of 330 MB. After pruning, the FPS improves to 6.22, marking a 36.93% increase in speed, while the model size reduces to 237 MB. Further quantization brings the FPS to 6.53, representing a 43.75% speed gain over the baseline, with a model size reduced to 229 MB. This optimization demonstrates both a significant increase in inference speed and a decrease in model size, making it more suitable for deployment on low-end CPUs.

Table 3: Inference Speed Comparison

Model	FPS	FPS Gain(%)	Size (MB)
Baseline	4.54	n/a	330
Pruned	6.22	+36.93	237
Pruned+Quantized	6.53	+43.75	229

6.2 Color Distribution Consistency

Our model demonstrates promising results in terms of the Color Distribution Consistency (CDC) score. This metric evaluates the consistency of predicted color distributions across the video. As shown in Figure 4, our model achieves a CDC score comparable to state-of-the-art methods. This suggests that the model can predict relatively consistent color distributions within individual frames, even though these predictions may vary in a spatio-temporal manner.

Method	CDC
AutoColor [22]	0.003734
Deolidiy	0.004901
DeepExemplar [40]	0.004006
DeepRemaster [11]	0.005098
TCVC [28]	0.003649
VCGAN [43]	0.005303
Diffusion [26]	0.003607
Ours	0.003193

Table 4: Comparison of CDC Score of different colorization pipelines on the DAVIS-test dataset. Lower is better.

6.3 Limitations Identified Through Qualitative Evaluation

As shown in Figure 4(c), the trained U-Net model tends to produce color patches that appear throughout each video sequence. These patches may vary in size and location across different frames, leading to a lack of visual consistency. The colorization process lacks spatio-temporal coherence, resulting in inconsistencies in color across consecutive frames. This effect manifests as fluctuations in

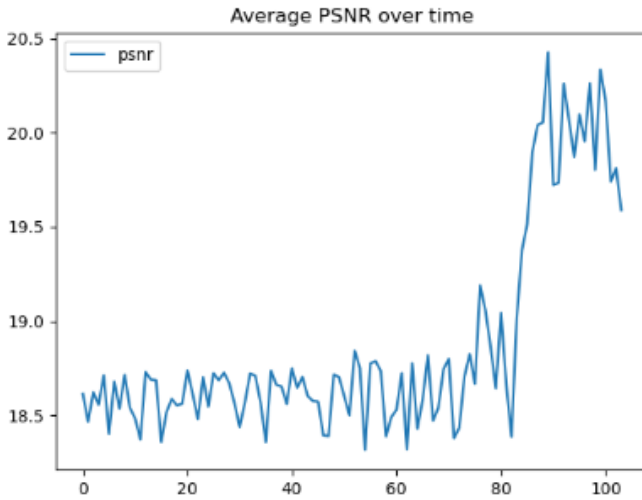


Figure 5: Frame Sequence plotted against PSNR.

the Peak Signal-to-Noise Ratio (PSNR) plot when plotted against frame number (see Figure 5). The significant variations in PSNR between frames highlight the inconsistency in color prediction across the video sequence.

7 Conclusion

In this work, we presented a real-time video colorization approach designed to operate efficiently on consumer-grade CPUs. By leveraging EfficientNet-B7 as the backbone of a U-Net architecture, followed by aggressive pruning and quantization, we achieved significant improvements in inference speed and memory efficiency while maintaining competitive performance in terms of color fidelity and temporal consistency. Our optimized model shows a 43.75% increase in inference speed and a 30.6% reduction in model size compared to the baseline, making it capable of colorizing videos at over 6 FPS on low-end hardware without dedicated acceleration.

While the model achieves a strong balance between speed and quality, challenges remain in ensuring perfect spatio-temporal coherence across video frames. Future work can focus on enhancing the model's temporal consistency and addressing occasional color patch artifacts without sacrificing performance. Our contributions pave the way for more accessible and efficient video colorization tools, especially in resource-constrained environments, and we hope our open-source release will spur further innovation in this space.

References

- [1] Ron Banner, Yury Nahshan, and Daniel Soudry. 2019. Post training 4-bit quantization of convolutional networks for rapid-deployment. *Advances in Neural Information Processing Systems* 32 (2019).
- [2] Jonathan Frankle and Michael Carbin. 2018. The lottery ticket hypothesis: Finding sparse, trainable neural networks. *arXiv preprint arXiv:1803.03635* (2018).
- [3] Yash Goyal, Tejas Khot, Douglas Summers-Stay, Dhruv Batra, and Devi Parikh. 2017. Making the V in VQA Matter: Elevating the Role of Image Understanding in Visual Question Answering. *arXiv:1612.00837* [cs.CV]
- [4] Yiwen Guo, Anbang Yao, and Yurong Chen. 2016. Dynamic network surgery for efficient dnns. *Advances in neural information processing systems* 29 (2016).
- [5] Song Han, Jeff Pool, John Tran, and William Dally. 2015. Learning both weights and connections for efficient neural network. *Advances in neural information processing systems* 28 (2015).
- [6] Babak Hassibi, David G Stork, and Gregory J Wolff. 1993. Optimal brain surgeon and general network pruning. In *IEEE international conference on neural networks*. IEEE, 293–299.
- [7] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 770–778.
- [8] Yihui He, Xiangyu Zhang, and Jian Sun. 2017. Channel pruning for accelerating very deep neural networks. In *Proceedings of the IEEE international conference on computer vision*. 1389–1397.
- [9] Yi-Chin Huang, Yi-Shin Tung, Jun-Cheng Chen, Sung-Wen Wang, and Ja-Ling Wu. 2005. An adaptive edge detection based colorization algorithm and its applications. In *Proceedings of the 13th Annual ACM International Conference on Multimedia* (<conf-loc>, <city>Hilton</city>, <country>Singapore</country>, </conf-loc>) (*MULTIMEDIA '05*). Association for Computing Machinery, New York, NY, USA, 351–354. <https://doi.org/10.1145/1101149.1101223>
- [10] Itay Hubara, Matthieu Courbariaux, Daniel Soudry, Ran El-Yaniv, and Yoshua Bengio. 2016. Binarized neural networks. *Advances in neural information processing systems* 29 (2016).
- [11] Satoshi Iizuka and Edgar Simo-Serra. 2020. DeepRemaster: Temporal Source-Reference Attention Networks for Comprehensive Video Enhancement. *arXiv:2009.08692* [cs.CV]
- [12] Satoshi Iizuka, Edgar Simo-Serra, and Hiroshi Ishikawa. 2016. Let there be color! joint end-to-end learning of global and local image priors for automatic image colorization with simultaneous classification. *ACM Trans. Graph.* 35, 4, Article 110 (jul 2016), 11 pages. <https://doi.org/10.1145/2897824.2925974>
- [13] Fakultit Informatik, Y. Bengio, Paolo Frasconi, and Jürgen Schmidhuber. 2003. Gradient Flow in Recurrent Nets: the Difficulty of Learning Long-Term Dependencies. *A Field Guide to Dynamical Recurrent Neural Networks* (03 2003).
- [14] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. 2018. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2704–2713.
- [15] Varun Jampani, Raghudeep Gadde, and Peter V. Gehler. 2017. Video Propagation Networks. *arXiv:1612.05478* [cs.CV]
- [16] Xiaozhong Ji, Boyuan Jiang, Donghao Luo, Guangpin Tao, Wenqing Chu, Zhifeng Xie, Chengjie Wang, and Ying Tai. 2022. Colorformer: Image colorization via color memory assisted hybrid-attention transformer. In *European Conference on Computer Vision*. Springer, 20–36.
- [17] Justin Johnson, Alexandre Alahi, and Li Fei-Fei. 2016. Perceptual Losses for Real-Time Style Transfer and Super-Resolution. *arXiv:1603.08155* [cs.CV]
- [18] Xiaoyang Kang, Tao Yang, Wenqi Ouyang, Peiran Ren, Lingzhi Li, and Xuansong Xie. 2023. Ddcolor: Towards photo-realistic image colorization via dual decoders. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 328–338.
- [19] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. 2012. Imagenet classification with deep convolutional neural networks. *Advances in neural information processing systems* 25 (2012).
- [20] Gustav Larsson, Michael Maire, and Gregory Shakhnarovich. 2017. Learning Representations for Automatic Colorization. *arXiv:1603.06668* [cs.CV]
- [21] Yann LeCun, John Denker, and Sara Solla. 1989. Optimal Brain Damage. In *Advances in Neural Information Processing Systems*, D. Touretzky (Ed.), Vol. 2. Morgan-Kaufmann. https://proceedings.neurips.cc/paper_files/paper/1989/file/6c9882bbac1c7093bd25041881277658-Paper.pdf
- [22] Chenyang Lei and Qifeng Chen. 2019. Fully Automatic Video Colorization with Self-Regularization and Diversity. *arXiv:1908.01311* [cs.CV]
- [23] Anat Levin, Dani Lischinski, and Yair Weiss. 2004. Colorization using optimization. *ACM Trans. Graph.* 23, 3 (aug 2004), 689–694. <https://doi.org/10.1145/1015706.1015780>
- [24] Fengfu Li, Bin Liu, Xiaoxing Wang, Bo Zhang, and Junchi Yan. 2016. Ternary weight networks. *arXiv preprint arXiv:1605.04711* (2016).
- [25] Hanzhao Lin and Rafael Ferreira. 2021. Automated Image Colorization Using Deep Learning. *CS229: Machine Learning Final Projects, Stanford University* (2021). <https://cs229.stanford.edu/proj2021spr/report2/81996670.pdf>
- [26] Hanyuan Liu, Minshan Xie, Jinbo Xing, Chengze Li, and Tien-Tsin Wong. 2023. Video Colorization with Pre-trained Text-to-Image Diffusion Models. *arXiv:2306.01732* [cs.CV]
- [27] Sifei Liu, Guangyu Zhong, Shalini De Mello, Jinwei Gu, Varun Jampani, Ming-Hsuan Yang, and Jan Kautz. 2018. Switchable Temporal Propagation Network. *arXiv:1804.08758* [cs.CV]
- [28] Yihao Liu, Hengyuan Zhao, Kelvin C. K. Chan, Xintao Wang, Chen Change Loy, Yu Qiao, and Chao Dong. 2021. Temporally Consistent Video Colorization with Deep Feature Propagation and Self-regularization Learning. *arXiv:2110.04562* [cs.CV]
- [29] Zhuang Liu, Jiaqiao Li, Zhiqiang Shen, Gao Huang, Shoumeng Yan, and Changshui Zhang. 2017. Learning efficient convolutional networks through network slimming. In *Proceedings of the IEEE international conference on computer vision*. 2736–2744.

- [30] I Loshchilov. 2017. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101* (2017).
- [31] Simone Meyer, Victor Cornillière, Abdelaziz Djelouah, Christopher Schroers, and Markus Gross. 2018. Deep video color propagation. *arXiv preprint arXiv:1808.03232* (2018).
- [32] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. 2019. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems* 32 (2019).
- [33] Gustav Grund Pihlgren, Konstantina Nikolaidou, Prakash Chandra Chhipa, Nosheen Abid, Rajkumar Saini, Fredrik Sandin, and Marcus Liwicki. 2023. A Systematic Performance Analysis of Deep Perceptual Loss Networks: Breaking Transfer Learning Conventions. *arXiv:2302.04032* [cs.CV]
- [34] Jordi Pont-Tuset, Federico Perazzi, Sergi Caelles, Pablo Arbeláez, Alexander Sorkine-Hornung, and Luc Van Gool. 2017. The 2017 DAVIS Challenge on Video Object Segmentation. *arXiv:1704.00675* (2017).
- [35] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. 2015. U-net: Convolutional networks for biomedical image segmentation. In *Medical image computing and computer-assisted intervention—MICCAI 2015: 18th international conference, Munich, Germany, October 5–9, 2015, proceedings, part III* 18. Springer, 234–241.
- [36] Karen Simonyan and Andrew Zisserman. 2015. Very Deep Convolutional Networks for Large-Scale Image Recognition. *arXiv:1409.1556* [cs.CV]
- [37] Mingxing Tan and Quoc V. Le. 2020. EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. *arXiv:1905.11946* [cs.LG]
- [38] Patricia Vitoria, Lara Raad, and Coloma Ballester. 2020. Chromagan: Adversarial picture colorization with semantic class distribution. In *Proceedings of the IEEE/CVF winter conference on applications of computer vision*. 2445–2454.
- [39] Liron Yatziv and Guillermo Sapiro. 2006. Fast image and video colorization using chrominance blending. *IEEE transactions on image processing : a publication of the IEEE Signal Processing Society* 15 (06 2006), 1120–9. <https://doi.org/10.1109/TIP.2005.864231>
- [40] Bo Zhang, Mingming He, Jing Liao, Pedro V. Sander, Lu Yuan, Amine Bermak, and Dong Chen. 2019. Deep Exemplar-based Video Colorization. *arXiv:1906.09909* [cs.CV]
- [41] Richard Zhang, Phillip Isola, and Alexei A. Efros. 2016. Colorful Image Colorization. *arXiv:1603.08511* [cs.CV]
- [42] Richard Zhang, Jun-Yan Zhu, Phillip Isola, Xinyang Geng, Angela S Lin, Tianhe Yu, and Alexei A Efros. 2017. Real-time user-guided image colorization with learned deep priors. *arXiv preprint arXiv:1705.02999* (2017).
- [43] Yuzhi Zhao, Lai-Man Po, Wing-Yin Yu, Yasar Abbas Ur Rehman, Mengyang Liu, Yujia Zhang, and Weifeng Ou. 2023. VCGAN: Video Colorization With Hybrid Generative Adversarial Network. *IEEE Transactions on Multimedia* 25 (2023), 3017–3032. <https://doi.org/10.1109/tmm.2022.3154600>
- [44] Bolei Zhou, Aditya Khosla, Agata Lapedriza, Antonio Torralba, and Aude Oliva. 2016. Places: An Image Database for Deep Scene Understanding. *arXiv:1610.02055* [cs.CV]